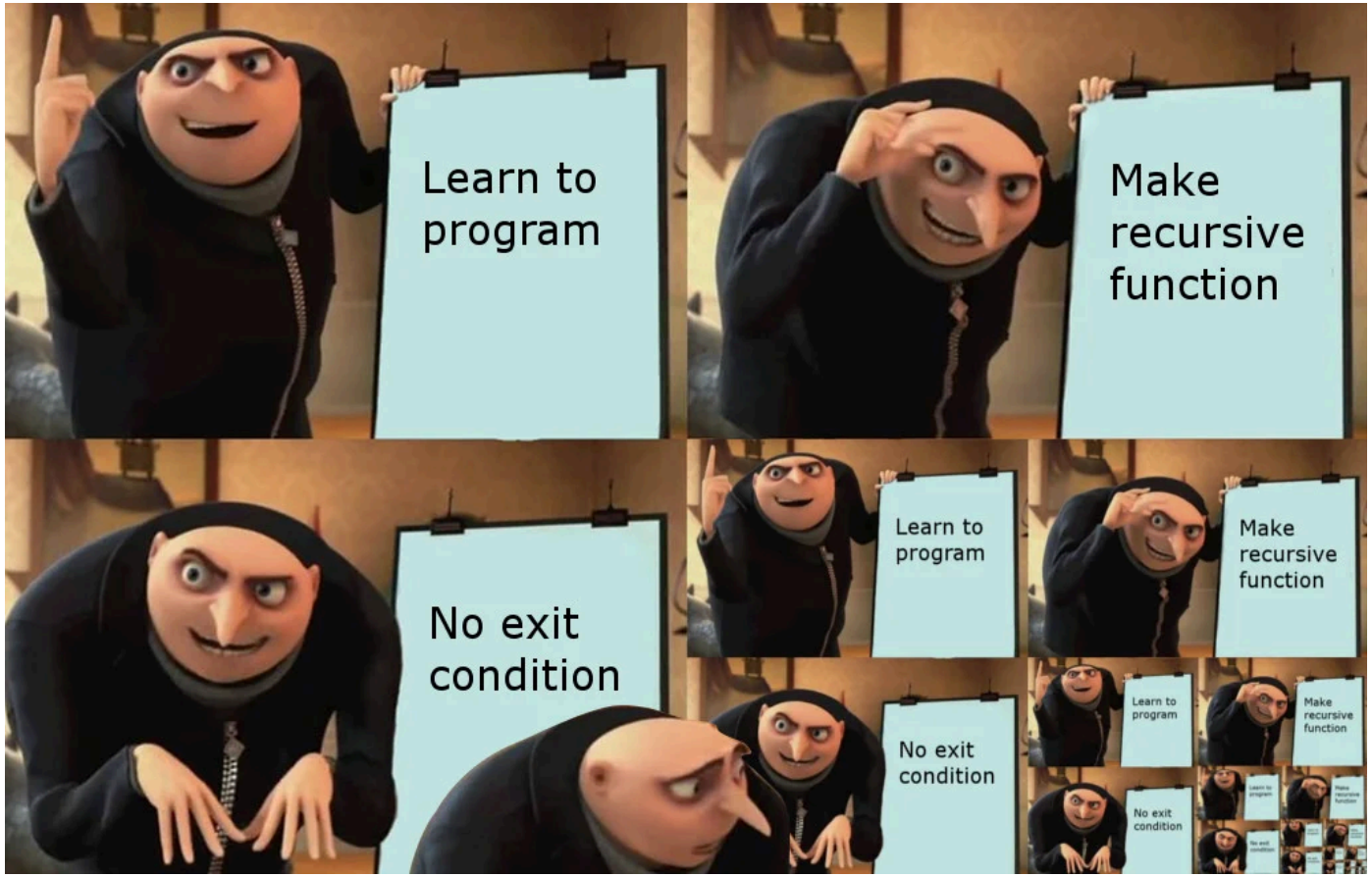
Rekurze

DELTA - Střední škola informatiky a ekonomie, s.r.o.

Ing. Luboš Zápotočný

07.04.2026

CC BY-NC-SA 4.0


Learn to program
Make recursive function
No exit condition
Learn to program
Make recursive function
No exit condition

Rekurze

Co je rekurze?

O rekurzi mluvíme, když funkce volá **sama sebe**

Aby rekurze někdy skončila, potřebujeme dvě věci:

1. **Základní případ** (base case) – kdy se rekurze zastaví?
2. **Rekurzivní krok** – jak se problém **zmenšuje** s každým voláním?

Co se stane, když základní případ chybí? **Stack overflow**

Co je rekurze?

Existuje i **nepřímá rekurze** – rekurzivní krok se provede až po několika volání jiných funkcí

```
void a(int n) {
    if (n <= 0) return;
    b(n - 1);
}

void b(int n) {
    if (n <= 0) return;
    a(n - 1);
}
```

Co je rekurze?

Většina rekurzivních algoritmů lze zapsat **iterativně** (pomocí cyklů)

```
int factorial_rec(int n) {
    if (n == 0) return 1;
    return n * factorial_rec(n - 1);
}

int factorial_iter(int n) {
    int result = 1;
    for (int i = 2; i <= n; i++)
        result *= i;
    return result;
}
```

Co je rekurze?

Rekurze je **jádro funkcionálního programování**

- Funkcionální jazyky (Haskell, Lisp, Erlang) nemají klasické cykly
- Místo `for`/`while` se používá rekurze
- Kompilátor optimalizuje **koncovou rekurzi na cyklus** (vysvětlení později)

Jednoduchý příklad – faktoriál

$$n! = n \cdot (n-1) \cdot (n-2) \cdot \ldots \cdot 2 \cdot 1$$

Rekurzivní definice:

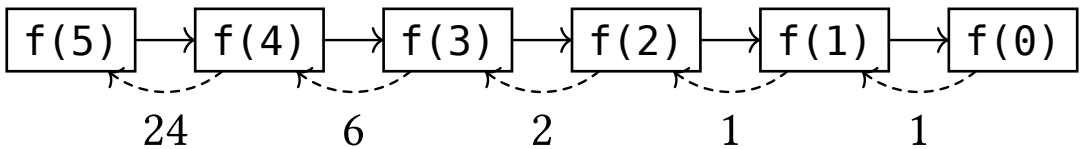
$$n! = \begin{cases} 1 & \text{pro } n = 0 \\ n \cdot (n-1)! & \text{pro } n > 0 \end{cases}$$

Jednoduchý příklad – faktoriál

```
int factorial(int n) {
    if (n == 0) return 1;         // base case
    return n * factorial(n - 1); // recursive step
}
```

Jednoduchý příklad – faktoriál

Volání `factorial(5)`:



Výsledek: $5 \cdot 24 = \mathbf{120}$

Rekurentní rovnice

Rekurentní rovnice

Rekurentní rovnice definuje hodnotu funkce pomocí **předchozích hodnot**

Jaké **výpočty** reprezentují následující rekurentní rovnice?

$$F(n) = \begin{cases} 0 & \text{pro } n = 0 \\ 1 & \text{pro } n = 1 \\ F(n-1) + F(n-2) & \text{pro } n \geq 2 \end{cases}$$

Fibonacciho číslo

Rekurentní rovnice

$$F(n) = \begin{cases} 1 & \text{pro } n = 0 \\ n \cdot F(n-1) & \text{pro } n > 0 \end{cases}$$

Faktoriál

$$T(n) = \begin{cases} 0 & \text{pro } n = 0 \\ 2 \cdot T(n-1) + 1 & \text{pro } n > 0 \end{cases}$$

Počet nutných přemístění při přesunu **Hanojské věže**

Rekurze a stack

Co se děje na stacku?

Každé volání funkce vytvoří **nový stack frame**

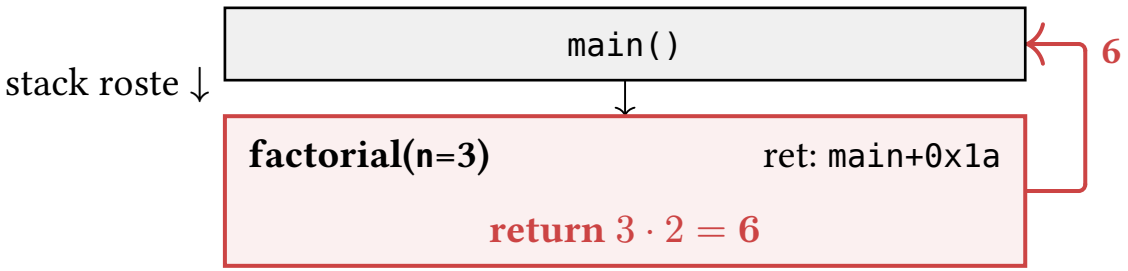
Při rekurzi se stack framy **vrší na sebe**

Po dosažení základního případu se framy **odvíjejí zpět**

Co se děje na stacku?

Volání `factorial(3)`:

```
main+0x18  call factorial
main+0x1a  ...
fact+0x00  int factorial(int n) {
fact+0x04    if (n == 0)
fact+0x0a      return 1;
fact+0x10    call factorial(n-1)
fact+0x14    n * <result>
fact+0x18    return
           }
```



Stack overflow

Stack overflow nastane, když je rekurze příliš hluboká

- Zásobník má **omezenou velikost** (typicky několik MB)
  - `ulimit -s`, výsledek v KB
- Při každém volání se alokuje nový frame
- Příliš mnoho vnořených volání ⇒ **přetečení zásobníku**

Proto je klíčové mít správný **základní případ**!

Výpis spojového seznamu od konce

Rekurzivní výpis od konce

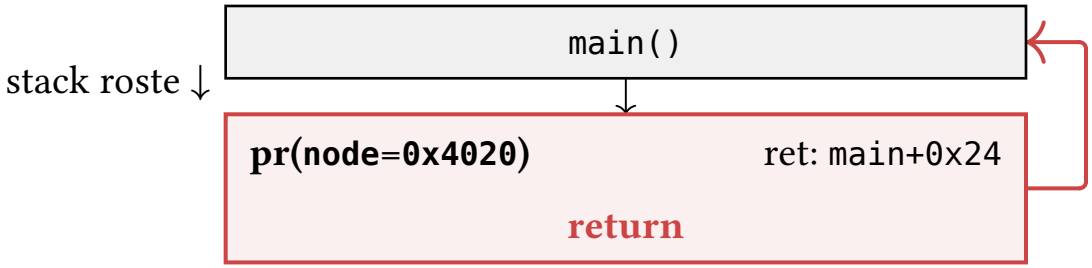
Jak vypsat spojový seznam **pozpátku** bez obousměrného zřetězení?

Rekurze! – nejdřív se zanoříme na konec, pak vypisujeme při návratu

```
void print_reverse(Node *node) {
    if (node == NULL) return;
    print_reverse(node->next);
    printf("%d ", node->data);
}
```

Rekurzivní výpis od konce

Adresa	data	next
0x4020	1	0x4030
0x4030	2	0x4040
0x4040	3	NULL




```
main+0x20  call pr(0x4020)
main+0x24  ...
pr+0x00    void pr(Node *node) {
pr+0x04      if (node == NULL)
pr+0x0a        return;
pr+0x0e      call pr(node->next)
pr+0x14      printf(node->data)
pr+0x1c      return
           }
```

Výstup: 3 2 1

Lokální proměnné na stacku

Každé volání funkce má **vlastní kopii** lokálních proměnných

```
int sum(Node *node) {
    if (node == NULL) return 0;
    int data = node->data;       // local variable
    int rest = sum(node->next);  // local variable
    return data + rest;
}
```

Ale u rekurze se to sčítá – **každé zanořené volání** alokuje nový frame s vlastním `data` a `rest`

Lokální proměnné na stacku

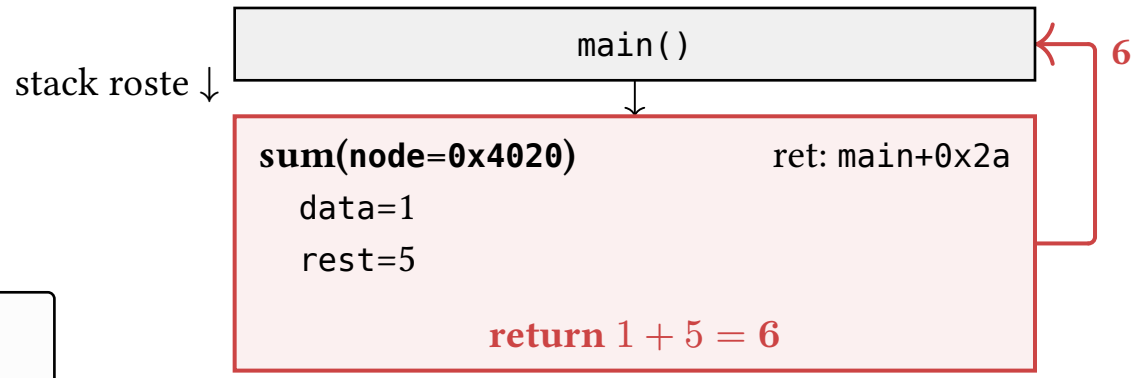
Každý frame tedy zabírá na stacku:

- **parametry**
- **lokální proměnné**
- **ret adresa**
- …

Lokální proměnné na stacku

Adresa	data	next
0x4020	1	0x4030
0x4030	2	0x4040
0x4040	3	NULL

```
main+0x28  call sum(0x4020)
main+0x2a  ...
sum+0x00   int sum(Node *node) {
sum+0x04     if (node == NULL)
sum+0x0a       return 0;
sum+0x0e     data = node->data
sum+0x14     rest = sum(node->next)
sum+0x1c     return data + rest
           }
```



Koncová rekurze

Co je koncová rekurze?

Co kdyby rekurzivní volání bylo to úplně **poslední**, co funkce udělá?

Pak nepotřebujeme aktuální stack frame – můžeme ho **přepoužít**

Tomu říkáme **koncová rekurze** (tail recursion)

Pozor: pokud po volání ještě něco počítáme
např. `fib(n-1) + fib(n-2)`, tak to koncová rekurze **není**

- potřebujeme udržet frame pro finální součet

Fibonacciho číslo – klasická rekurze

```
unsigned long long fib(int n) {
    if (n < 2) return n;
    return fib(n - 1) + fib(n - 2);
}
```

Fibonacciho číslo – koncová rekurze

```
unsigned long long fib_tail(int a, int b, int n) {
    if (n == 0) return a;
    return fib_tail(b, a + b, n - 1);
}

unsigned long long fib(int n) {
    return fib_tail(0, 1, n);
}
```

Koncová rekurze a stack

Při koncové rekurzi může být původní stack frame **přepoužit**

Pouze se změní registry na hodnoty nově předaných parametrů a skočí se na začátek původního rámce

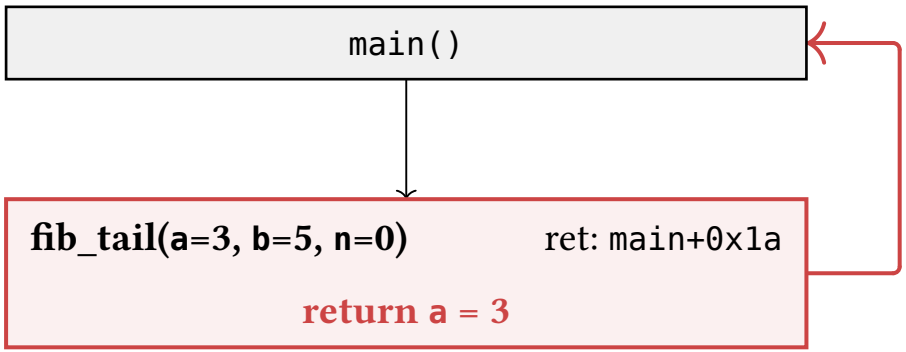
⇒ **Žádné nové stack framy** ⇒ **konstantní** spotřeba paměti na stacku

Kompilátor **může** tuto optimalizaci provést automaticky (tail call optimization) – v C závisí na optimalizační úrovni (`-O2`)

Koncová rekurze a stack

`fib_tail(0, 1, 4)` – jeden frame se přepoužívá:

```
fib_tail(int a, int b, int n) {
  if (n == 0)
    return a;
  a' = b
  b' = a + b
  n' = n - 1
  jump fib_tail
}
```



Stále jen **1 stack frame** – $\mathcal{O}(1)$ paměti na stacku

Malé intro do assembly

Malé intro do assembly

Assembly = strojový kód čitelný pro lidi – každý řádek je **jedna instrukce procesoru**

C kód

```c
int add(int a, int b) {
    int c = a + b;
    return c;
}
```

Co z toho udělá kompilátor

```asm
add:
  lea eax, [rdi+rsi]  ; eax = a + b
  ret                 ; return eax
```

Registry (`eax`, `rdi`, `rsi`, ...) = proměnné procesoru

`rdi` = 1. parametr, `rsi` = 2. parametr, `eax` = návratová hodnota

Malé intro do assembly

Trochu složitější – **volání funkce**:

```c
int square(int x) {
    return x * x;
}

int sum_of_squares(int a, int b) {
    int sa = square(a);
    int sb = square(b);
    return sa + sb;
}
```

```asm
sum_of_squares:
  push rbx        ; save rbx to stack
  mov  ebx, esi   ; rbx = b (save for later)
  call square     ; square(a) -> result in eax
  mov  edi, ebx   ; set b as parameter
  call square     ; square(b) -> result in eax
  add  eax, ebx   ; eax = sa + sb
  pop  rbx        ; restore rbx from stack
  ret
```

`push`/`pop` = schovat/obnovit hodnotu přes volání funkce

`call` = skoč do funkce (a zapamatuj si kam se vrátit)

Koncová rekurze v assembly

Klasická rekurze – C kód vs. assembly (`gcc -O2`):

```c
unsigned long long fib(int n) {
    if (n < 2) return n;
    return fib(n-1) + fib(n-2);
}
```

```asm
fib:
  push ...            ; save registers
  sub  rsp, 120       ; reserve 120B on stack!
  ...                 ; note: *
  call fib            ; fib(n-1)
  ...                 ; note: *
  call fib            ; fib(n-2)
  add  eax, ebx       ; fib(n-1) + fib(n-2)
  ...                 ; note: *
  add  rsp, 120       ; free stack space
  pop  ...            ; restore registers
  ret
```

* celkem ~300 řádků – `-O2` částečně rozbalil rekurzi do cyklů a větvení, ale `call fib` tam stále zůstává (bez optimalizací by kód byl kratší, ale pomalejší)

Koncová rekurze v assembly

Koncová rekurze – C kód vs. assembly (`gcc -O2`):

```c
unsigned long long fib_tail(
    int a, int b, int n) {
    if (n == 0) return a;
    return fib_tail(b, a+b, n-1);
}
```

```asm
fib_tail:
                          ; no push/sub!
.L2:
  lea eax, [rdi+rsi]      ; eax = a + b
  mov edi, esi            ; a = b
  sub edx, 1              ; n = n - 1
  jne .L2                 ; if n != 0: loop
  ret
```

Žádný `push`, žádný `call` – kompilátor rekurzi **přepsal na cyklus**

Koncová rekurze v assembly

	Klasická	**Koncová**
Stack operace	`push` + `sub rsp, 120`	*žádné*
Volání	2× `call fib`	`jne` (pouhý skok)
Stack framů	$\mathcal{O}(n)$	$\mathcal{O}(1)$
Délka assembly	~300 řádků	5 řádků

Koncová rekurze v assembly

Jak se k tomu dostat sami?

```
gcc -O2 -S -masm=intel fib-tail.c
```

`-S`	Generuj assembly místo binárky (výstup: `.s` soubor)
`-O2`	Zapni optimalizace – bez nich kompilátor tail call **neoptimalizuje**
`-masm=intel`	Intel syntax (`mov eax, 1`) místo AT&T (`movl $1, %eax`)

Koncová rekurze v assembly

Výsledek si lze prohlédnout v libovolném textovém editoru, nebo online na godbolt.org (Compiler Explorer)

Shrnutí

- **Rekurze** = funkce volající sama sebe (základní případ + rekurzivní krok)
- Každé volání vytvoří **nový stack frame** – pozor na stack overflow
- **Koncová rekurze** umožňuje přepoužít stack frame $\Rightarrow \mathcal{O}(1)$ paměti
- Rekurzi lze (většinou) přepsat na **iterativní řešení**